



University of Global Village (UGV), Barishal

Department of Computer Science and Engineering

Polymorphism in JAVA

Compile-Time and Runtime Polymorphism in Java

Method Overloading & Method Overriding

OVERLOADING • COMPILE-TIME

OVERRIDING • RUNTIME

Course Teacher
MD. MEHADI HASAN

Lecturer

Department of CSE
University of Global Village(UGV), Barishal

Introduction to Polymorphism

Polymorphism is one of the fundamental concepts of [Object-Oriented Programming](#) (OOP) that allows a single entity to take multiple forms.

It enables the same method or interface to behave differently depending on the object involved. Polymorphism improves code flexibility, reusability, and maintainability.

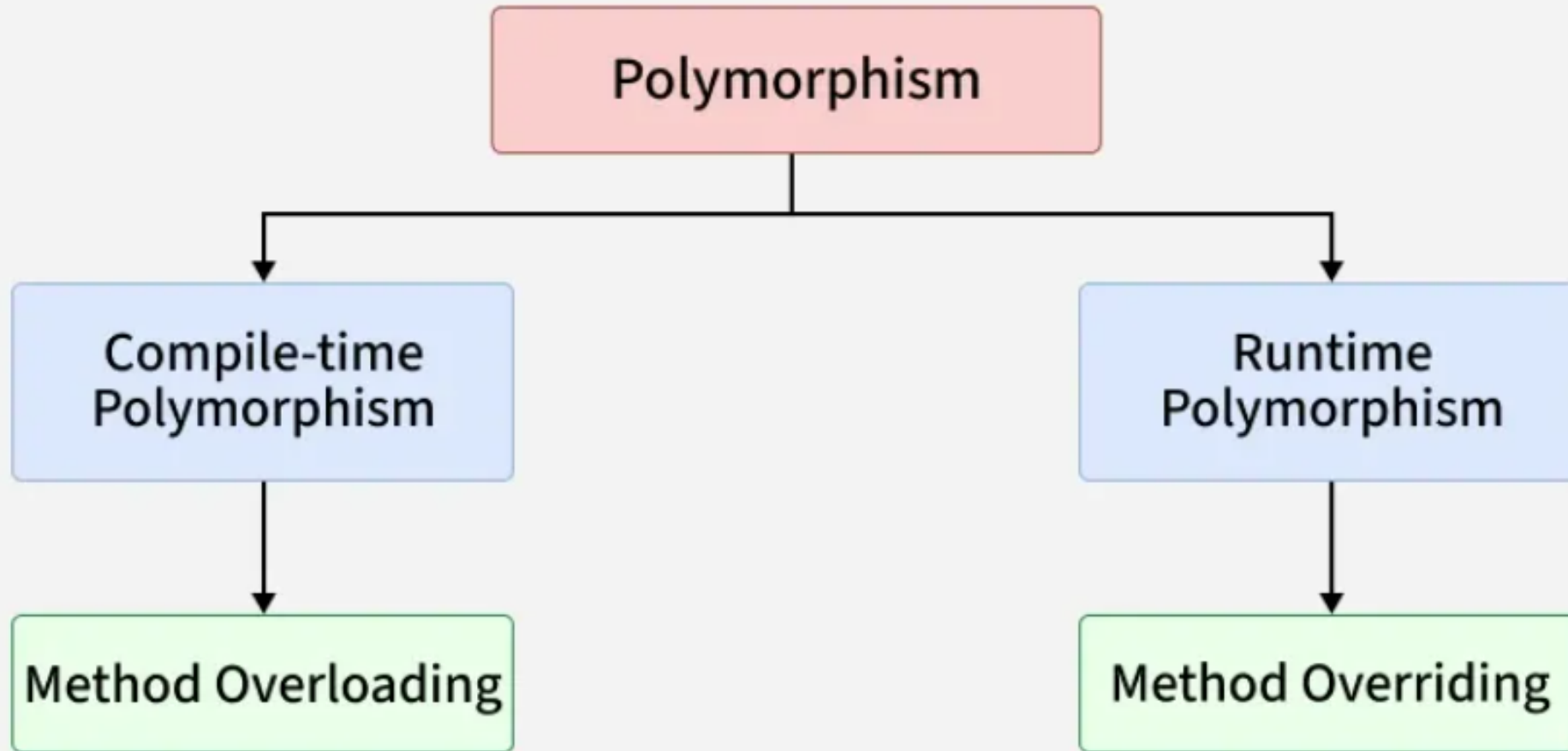
- Polymorphism means "many forms".
- It allows the same method to exhibit different behaviors.
- It is achieved through method overloading and method overriding.

Example of Polymorphism

```
class Animal {  
    public void animalSound() {  
        System.out.println("The animal makes a sound");  
    }  
}  
  
class Pig extends Animal {  
    public void animalSound() {  
        System.out.println("The pig says: wee wee");  
    }  
}  
  
class Dog extends Animal {  
    public void animalSound() {  
        System.out.println("The dog says: bow wow");  
    }  
}
```

Types of Polymorphism in Java

In Java Polymorphism is mainly divided into two types:



1. Compile-Time Polymorphism

Compile-time polymorphism also known as static polymorphism, occurs when the method to be executed is determined during compilation. It is primarily achieved through method overloading.

- Method selection occurs at compile time.
- Improves readability and flexibility.

JAVA Method Overloading

Method overloading allows multiple methods with the same name but different parameter lists within the same class. The compiler chooses the appropriate method based on the arguments passed.

Syntax:

```
class Demo {  
    void show(int a) {  
    }  
    void show(int a, int b) {  
    }  
}
```

1. Changing the Number of Parameters

Method overloading can be achieved by changing the number of parameters when passing to different methods.

```
class Product{

    // Multiplying two integer values
    public int multiply(int a, int b){

        int prod = a * b;
        return prod;
    }

    // Multiplying three integer values
    public int multiply(int a, int b, int c){

        int prod = a * b * c;
        return prod;
    }
}
```

Output

```
Product of the two integer value: 2
Product of the three integer value: 6
```

```
class Geeks{

    public static void main(String[] args)
    {

        Product ob = new Product();

        // Calling method to Multiply 2 numbers
        int prod1 = ob.multiply(1, 2);

        // Printing Product of 2 numbers
        System.out.println(
            "Product of the two integer value: " + prod1);

        // Calling method to multiply 3 numbers
        int prod2 = ob.multiply(1, 2, 3);

        // Printing product of 3 numbers
        System.out.println(
            "Product of the three integer value: " + prod2);

    }
}
```

2. Changing Data Types of Parameters

In many cases, methods can be considered overloaded if they have the same name but have different parameter types, methods are considered to be overloaded.

```
class Product{  
  
    public int prod(int a, int b, int c){  
        return a * b * c;  
    }  
  
    public double prod(double a, double b, double c){  
        return a * b * c;  
    }  
}  
  
public class Geeks {  
    public static void main(String[] args){  
  
        Product p = new Product();  
        System.out.println(p.prod(1, 2, 3));  
        System.out.println(p.prod(1.0, 2.0, 3.0));  
    }  
}
```

Output

```
6  
6.0
```

3. Changing the Order of Parameters

Method overloading can also be implemented by rearranging the parameters of two or more overloaded methods.

```
class Student {  
  
    public void studentId(String name, int rollNo){  
  
        System.out.println("Name: " + name  
                             + ", Roll-No: " + rollNo);  
    }  
    public void studentId(int rollNo, String name){  
  
        System.out.println("Roll-No: " + rollNo  
                             + ", Name: " + name);  
    }  
}
```

```
public class Geeks{  
  
    public static void main(String[] args){  
  
        Student s = new Student();  
        s.studentId("Sweta", 1);  
        s.studentId(2, "Gudly");  
    }  
}
```

Output

```
Name: Sweta, Roll-No: 1  
Roll-No: 2, Name: Gudly
```


2. Runtime Polymorphism

Runtime polymorphism also known as [dynamic polymorphism](#), occurs when the method call is resolved during program execution. It is achieved through method overriding.

- Method selection occurs at runtime.
- Depends on the actual object type.

Method Overriding

[Method overriding](#) occurs when a subclass provides its own implementation of a method already defined in its superclass.

That means if the same method is defined in both the superclass and the subclass, then the method of the subclass class overrides the method of the superclass. This is known as method overriding.

Syntax:

```
class Parent {  
    void print() {  
    }  
}  
  
class Child extends Parent {  
    @Override  
    void print() {  
    }  
}
```

Example 1: Method Overriding

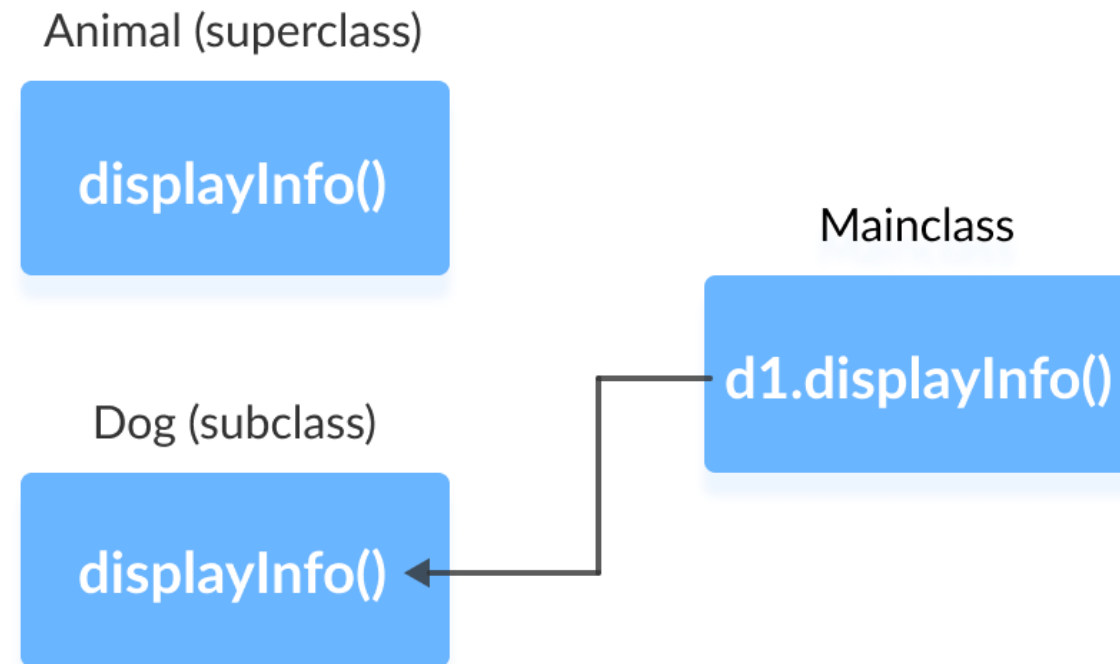
```
class Animal {  
    public void displayInfo() {  
        System.out.println("I am an animal.");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    public void displayInfo() {  
        System.out.println("I am a dog.");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Dog d1 = new Dog();  
        d1.displayInfo();  
    }  
}
```

Output:

I am a dog.

In the above program, the `displayInfo()` method is present in both the *Animal* superclass and the *Dog* subclass.

When we call `displayInfo()` using the `d1` object (object of the subclass), the method inside the subclass *Dog* is called. The `displayInfo()` method of the subclass overrides the same method of the superclass.



Java Overriding Rules

- Both the superclass and the subclass must have the same method name, the same return type and the same parameter list.
- We cannot override the method declared as final and static.
- We should always override abstract methods of the superclass (will be discussed in later tutorials).

super Keyword in Java Overriding

A common question that arises while performing overriding in Java is:

Can we access the method of the superclass after overriding?

Well, the answer is **Yes**. To access the method of the superclass from the subclass, we use the super keyword.

It eliminates the confusion between superclasses() and subclasses() that have methods with the same name.

- It can be used in two main ways:
- To access attributes and methods from the parent class
- To call the parent class constructor

Example 2: Use of super Keyword

```
class Animal {  
    public void animalSound() {  
        System.out.println("The animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    public void animalSound() {  
        super.animalSound(); // Call the parent method  
        System.out.println("The dog says: bow wow");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog myDog = new Dog();  
        myDog.animalSound();  
    }  
}
```

Output:

```
The animal makes a sound  
The dog says: bow wow
```

Access Parent Attributes

You can also use **super** to access an attribute from the parent class if they have an attribute with the same name:

```
class Animal {
    String type = "Animal";
}

class Dog extends Animal {
    String type = "Dog";

    public void printType() {
        System.out.println(super.type); // Access parent attribute
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.printType();
    }
}
```

Output:

Animal

References

- [1] <https://www.geeksforgeeks.org/java/polymorphism-in-java/>
- [2] https://www.tutorialspoint.com/java/java_polymorphism.htm
- [3] <https://www.programiz.com/java-programming/method-overriding>